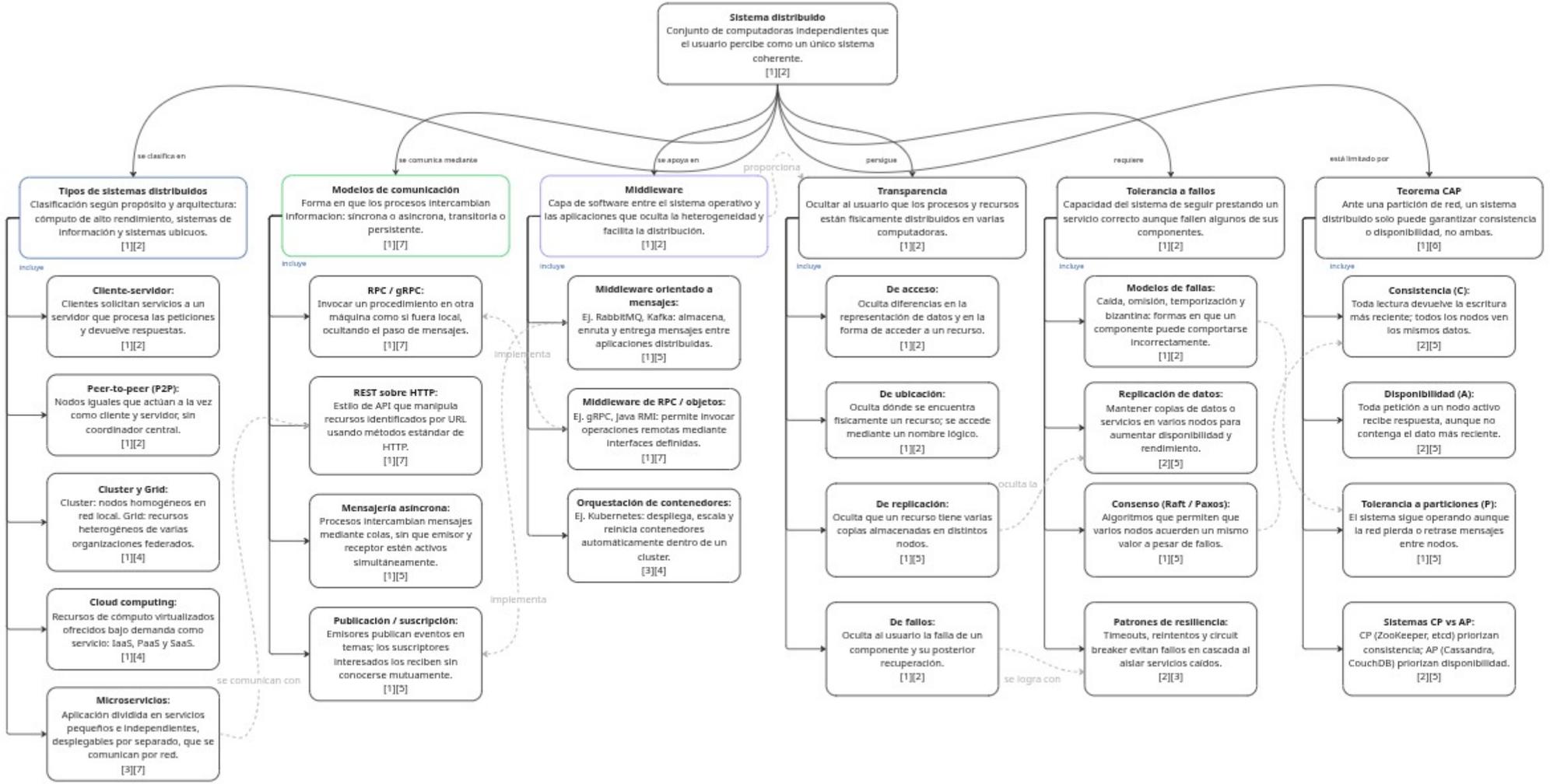

Sistema distribuido
Conjunto de computadoras independientes que el usuario percibe como un único sistema coherente.
[1][2]
se clasifica en
se comunica mediante
se apoya en
proporciona
persigue
requiere
está limitado por
Tipos de sistemas distribuidos
Clasificación según propósito y arquitectura: cómputo de alto rendimiento, sistemas de información y sistemas ubicuos.
[1][2]
incluye
Cliente-servidor:
Clientes solicitan servicios a un servidor que procesa las peticiones y devuelve respuestas.
[1][2]
Peer-to-peer (P2P):
Nodos iguales que actúan a la vez como cliente y servidor, sin coordinador central.
[1][2]
Cluster y Grid:
Cluster: nodos homogéneos en red local. Grid: recursos heterogéneos de varias organizaciones federados.
[1][4]
Cloud computing:
Recursos de cómputo virtualizados ofrecidos bajo demanda como servicio: IaaS, PaaS y SaaS.
[1][4]
Microservicios:
Aplicación dividida en servicios pequeños e independientes, desplegables por separado, que se comunican por red.
[3][7]
Modelos de comunicación
Forma en que los procesos intercambian información: síncrona o asíncrona, transitoria o persistente.
[1][7]
incluye
RPC / gRPC:
Invocar un procedimiento en otra máquina como si fuera local, ocultando el paso de mensajes.
[1][7]
REST sobre HTTP:
Estilo de API que manipula recursos identificados por URL usando métodos estándar de HTTP.
[1][7]
Mensajería asíncrona:
Procesos intercambian mensajes mediante colas, sin que emisor y receptor estén activos simultáneamente.
[1][5]
Publicación / suscripción:
Emisores publican eventos en temas; los suscriptores interesados los reciben sin conocerse mutuamente.
[1][5]
se comunican con
Middleware
Capa de software entre el sistema operativo y las aplicaciones que oculta la heterogeneidad y facilita la distribución.
[1][2]
incluye
Middleware orientado a mensajes:
Ej. RabbitMQ, Kafka: almacena, enruta y entrega mensajes entre aplicaciones distribuidas.
[1][5]
Middleware de RPC / objetos:
Ej. gRPC, Java RMI: permite invocar operaciones remotas mediante interfaces definidas.
[1][7]
Orquestación de contenedores:
Ej. Kubernetes: despliega, escala y reinicia contenedores automáticamente dentro de un cluster.
[3][4]
implementa
implementa
Transparencia
Ocultar al usuario que los procesos y recursos están físicamente distribuidos en varias computadoras.
[1][2]
incluye
De acceso:
Oculta diferencias en la representación de datos y en la forma de acceder a un recurso.
[1][2]
De ubicación:
Oculta dónde se encuentra físicamente un recurso; se accede mediante un nombre lógico.
[1][2]
De replicación:
Oculta que un recurso tiene varias copias almacenadas en distintos nodos.
[1][5]
De fallos:
Oculta al usuario la falla de un componente y su posterior recuperación.
[1][2]
oculta la
se logra con
Tolerancia a fallos
Capacidad del sistema de seguir prestando un servicio correcto aunque fallen algunos de sus componentes.
[1][2]
incluye
Modelos de fallas:
Caída, omisión, temporización y bizantina: formas en que un componente puede comportarse incorrectamente.
[1][2]
Replicación de datos:
Mantener copias de datos o servicios en varios nodos para aumentar disponibilidad y rendimiento.
[2][5]
Consenso (Raft / Paxos):
Algoritmos que permiten que varios nodos acuerden un mismo valor a pesar de fallos.
[1][5]
Patrones de resiliencia:
Timeouts, reintentos y circuit breaker evitan fallos en cascada al aislar servicios caídos.
[2][3]
Teorema CAP
Ante una partición de red, un sistema distribuido solo puede garantizar consistencia o disponibilidad, no ambas.
[1][6]
incluye
Consistencia (C):
Toda lectura devuelve la escritura más reciente; todos los nodos ven los mismos datos.
[2][5]
Disponibilidad (A):
Toda petición a un nodo activo recibe respuesta, aunque no contenga el dato más reciente.
[2][5]
Tolerancia a particiones (P):
El sistema sigue operando aunque la red pierda o retrase mensajes entre nodos.
[1][5]
Sistemas CP vs AP:
CP (ZooKeeper, etcd) priorizan consistencia; AP (Cassandra, CouchDB) priorizan disponibilidad.
[2][5]